N2_GCPS — Aplicativo Contador Flutter

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

Versão 1.0

Campo	Descrição
Projeto	N2_GCPS — Aplicativo Contador Flutter
Gerente de Projeto	Vinicius Batista Condes
Gerente/Responsável de GCS-DevOps	Vinicius Batista Condes (conta GitHub: Klebynhow)
Fábrica de Software / Squad	Individual — Vinicius Batista Condes (revisor: conta Vinicius-Batista-Condes)
Repositório principal	https://github.com/Klebynhow/GCPS-N2

Histórico de Revisões

Data	Versão	Descrição	Autor
27/10/2026	1.0	Elaboração do documento com enfoque em práticas DevOps (CI/CD) para o projeto N2_GCPS.	Vinicius Batista Condes

Sumário

Sumário 2
1. Introdução 3
1.1 Finalidade 3
1.2 Escopo 3
1.3 Definições, Acrônimos e Abreviações 3
1.4 Referências 3
1.5 Visão Geral 4
2. Cultura DevOps e Papéis na Gerência de Configuração 4
3. Ferramentas, Ambientes e Infraestrutura 4
3.1 Ferramentas do Pipeline 4
3.2 Ambientes e Infraestrutura 5
3.3 Estratégia de Branching e Versionamento 5
4. Identificação da Configuração 6
4.1 Itens de Configuração 6
4.2 Convenção de Nomenclatura 6
4.3 Estrutura do Repositório 7
5. Integração Contínua (CI) 7
5.1 Etapas do Pipeline de CI 7
5.2 Política de Pull Request e Revisão de Código 7
5.3 Critérios de Qualidade (Quality Gate) 8
6. Entrega e Implantação Contínua (CD) 8
6.1 Etapas do Pipeline de CD 8
6.2 Estratégias de Implantação 8
6.3 Rollback e Recuperação 9
7. Infraestrutura como Código (IaC) 9
7.1 Versionamento de Infraestrutura 9
7.2 Ambientes Efêmeros 9
8. Controle de Configuração e Mudança 9
8.1 Processo de Solicitação de Mudança 9
8.2 Comitê de Controle de Mudança (CCM) 9
9. Baselines e Releases 9
9.1 Geração Automática de Baseline 10
9.2 Notas de Versão (Release Notes) 10
10. Estimativa do Status de Configuração 10
10.1 Monitoramento e Observabilidade 10
10.2 Métricas DORA 10
10.3 Relatórios e Auditorias 10
11. Marcos 10
12. Treinamento e Recursos 10
13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros 11
14. Quadro-Resumo do Pipeline CI/CD 11

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) que serão executadas ao longo do ciclo de vida do produto, com ênfase em práticas DevOps — em particular Integração Contínua (CI), Entrega/Implantação Contínua (CD), Infraestrutura como Código (IaC) e automação de pipelines. As atividades envolvem identificar os itens de configuração, manter sua integridade e controlar sistematicamente as mudanças, com o mínimo de burocracia possível e o máximo de automação.

1.1 Finalidade

Estabelecer um padrão a ser seguido pela equipe/individuo, garantindo rastreabilidade, versionamento e controle de mudanças do produto, apoiado por pipelines automatizados de build, teste, integração e implantação.

1.2 Escopo

Este PGC destina-se ao desenvolvedor individual Vinicius Batista Condes (conta Klebynhow), responsável pelo desenvolvimento e pela operação do aplicativo N2_GCPS: um app Flutter (contador de cliques, projeto padrão do framework) com alvos web e Android. O projeto tem finalidade didática: demonstrar, na prática, conceitos de GCS e CI/CD. O plano abrange o código-fonte Dart e os testes, as configurações Android e web, os modelos de colaboração (.github) e a configuração do repositório no GitHub (proteção da main, merge, ambientes e revisão).

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
CI	Integração Contínua (Continuous Integration)
CD	Entrega/Implantação Contínua (Continuous Delivery / Deployment)
IaC	Infraestrutura como Código (Infrastructure as Code)
CCM	Comitê de Controle de Mudanças
Baseline	Conjunto de itens de configuração que atingiram um estado comprovado de estabilidade
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação
Trunk-based Development	Estratégia de branching em que os desenvolvedores integram na branch principal com alta frequência
SemVer	Versionamento Semântico (MAJOR.MINOR.PATCH) - 1.0.0.
DORA Metrics	Métricas de desempenho DevOps (Lead Time, Deployment Frequency, MTTR, Change Failure Rate)

1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM) — adaptado.

Material da Aula 1 do projeto ongconecta (repositório de referência da disciplina). Documentação oficial do Flutter (docs.flutter.dev) e do GitHub (docs.github.com): branch protection, environments e Pull Requests.

1.5 Visão Geral

Diferente de processos tradicionais em que a GCS é predominantemente manual e concentrada em marcos formais, neste plano a GCS é tratada como parte constante do pipeline de entrega: cada commit integrado, testado e implantado automaticamente gera evidências de rastreabilidade (logs de pipeline, tags de versão, artefatos publicados), reduzindo o esforço manual e o tempo entre a mudança e sua entrega em produção.

2. Cultura DevOps e Papéis na Gerência de Configuração

A GCS em um contexto **DevOps é responsabilidade compartilhada entre desenvolvimento e operações ("Você constrói, você executa" — you build it, you run it), apoiada por automação.** Os papéis abaixo descrevem responsabilidades típicas, que podem ser exercidas por uma mesma pessoa em squads pequenas.

Papel	Responsabilidades
Gerente de Projeto / Product Owner	Prioriza o backlog, autoriza mudanças de maior impacto e acompanha indicadores de entrega (DORA Metrics).
Engenheiro DevOps / Release Engineer	Projeta e mantém os pipelines de CI/CD, define a estratégia de *branching* e versionamento, administra o registro de artefatos/imagens e a IaC.
Desenvolvedor(a)	Integra código com frequência na branch principal, escreve testes automatizados, revisa Pull Requests e é corresponsável pela qualidade do que é implantado.
SRE / Operações *Site Reliability Engineering*	Monitora ambientes de produção, define SLIs/SLOs, participa de postmortems e realiza rollback quando necessário. - *Service Level Indicators (SLIs) and Service Level Objectives (SLOs).*
QA / Automação de Testes	Mantém a suíte de testes automatizados (unitários, integração, aceitação) executada nos pipelines de CI e CD.
Segurança (DevSecOps)	Define políticas de análise estática (SAST), análise de dependências (SCA) e gestão de segredos integradas ao pipeline.
Comitê de Controle de Mudanças (CCM)	Atua apenas em mudanças de alto risco/impacto (ex.: mudanças em produção fora do fluxo padrão); mudanças de rotina seguem o fluxo automatizado de PR + pipeline.

No N2_GCPS os papéis acima são acumulados por uma única pessoa, Vinicius Batista Condes. Para cumprir a regra dos quatro olhos, existem duas contas GitHub: Klebynhow (proprietário, desenvolvedor e revisor obrigatório do ambiente production) e Vinicius-Batista-Condes (par de revisão, com permissão Write, que aprova os Pull Requests). O GitHub não permite aprovar o próprio PR, por isso a revisão é feita pela segunda conta.

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Ferramentas adotadas no projeto N2_GCPS. Itens marcados como "planejado" serão implementados nas Aulas 2 e 3.

Categoria	Ferramenta(s) adotada(s)	Finalidade
Controle de versão	Git + GitHub	Versionamento do código, Pull Requests, proteção da main e modelos de colaboração.
Orquestração de CI/CD	GitHub Actions (planejado, Aulas 2 e 3)	Build, análise, testes e implantação automatizados.
Empacotamento	Flutter SDK (flutter build web / apk)	Geração dos artefatos web e Android a partir do código.
Registro de artefatos	GitHub Releases e artifacts do Actions	Armazenamento e versionamento dos artefatos gerados (APK, build web).
Infraestrutura como Código	Não aplicável	Não há infraestrutura própria; a configuração do GitHub é documentada neste plano (ver 7).
Orquestração de containers	Não utilizado	O app não é conteinerizado.
Gestão de segredos	GitHub Secrets (repositório e environments)	Armazenamento de credenciais usadas pelos pipelines; nenhuma no código.
Monitoramento e Observabilidade	Não aplicável nesta fase	O app não tem backend; a saúde é acompanhada por workflows, deployments e smoke tests.
Gestão de dependências / SCA – *Software Component Analisys*	Dependabot (ecossistema pub) e flutter pub outdated (planejado)	Identificação de dependências desatualizadas ou vulneráveis.
Rastreamento de tarefas	GitHub Issues (modelos funcionalidade e defeito)	Vinculação de commits e PRs a itens de trabalho (Closes #).

3.2 Ambientes e Infraestrutura

Como o projeto é didático e não possui servidores próprios, os ambientes de homologação e produção são representados por GitHub Environments, que controlam quem aprova e de quais branches se pode implantar.

Ambiente	Finalidade	Gatilho de promoção
Desenvolvimento	Máquina local do desenvolvedor (emulador Android ou Chrome); no CI, runners efêmeros do GitHub Actions.	Push em branch de feature e abertura de Pull Request.
Homologação / Staging	GitHub Environment staging, restrito a branches protegidas (main). Validação do build antes da produção.	Merge na main (PR aprovado e CI verde).
Produção	GitHub Environment production, restrito a branches protegidas, com revisor obrigatório (Klebynhow) e "Prevent self-review" desmarcado.	Aprovação manual do revisor no gate de release, após validação em staging.

Não há DNS, IPs ou banco de dados a detalhar. Credenciais, quando necessárias, ficam em GitHub Secrets por environment e nunca no código.

3.3 Estratégia de Branching e Versionamento

Adota-se o GitHub Flow (variante de Trunk-Based Development), com branches de vida curta e merge exclusivamente por squash:

- main — sempre implantável. Protegida: Pull Request obrigatório com 1 aprovação, aprovações descartadas quando há novos commits, histórico linear e sem possibilidade de bypass (nem para administradores). Status checks obrigatórios serão habilitados na Aula 2.
- feature/<issue>-<descrição-curta> (ou fix/<issue>-<descrição-curta> para defeitos) — branches de curta duração, uma por mudança. São apagadas automaticamente após o merge.
- release/x.y — não utilizada. As versões são marcadas por tags vX.Y.Z geradas a partir da main.
- hotfix/<descrição> — correções emergenciais seguem o mesmo fluxo de PR (a main não aceita push direto) e acionam o CCM (ver 8.2).

Versionamento: Versionamento Semântico (MAJOR.MINOR.PATCH) e Conventional Commits (feat, fix, chore, docs, refactor...). Só é permitido squash merge, com o título e a descrição do PR como mensagem do commit; por isso o título do PR segue Conventional Commits e é dele que o pipeline da Aula 3 calculará a próxima versão. A versão do app fica em pubspec.yaml (campo version, ex.: 1.0.0+1).

4. Identificação da Configuração

4.1 Itens de Configuração

Além do código-fonte, em um contexto DevOps os seguintes artefatos também são itens de configuração formais, versionados no mesmo repositório (ou repositórios compartilhados):

- Código-fonte Dart (lib/) e testes automatizados (test/)
- Definições de pipeline como código (.github/workflows — a serem criadas nas Aulas 2 e 3)
- Dockerfiles e manifests de containers — não aplicável (app Flutter não conteinerizado)
- Infraestrutura como Código — não aplicável; a configuração do repositório e dos ambientes está documentada neste plano
- Arquivos de configuração: pubspec.yaml, analysis_options.yaml, .metadata, configurações Android (android/, AndroidManifest.xml, *.gradle.kts) e web (web/index.html, manifest.json). O arquivo .env.example lista os nomes das variáveis de ambiente, sem valores; segredos ficam em GitHub Secrets
- Controle de dependências e do repositório: pubspec.lock (versões exatas), .gitignore e .gitattributes (* text=auto eol=lf)
- Scripts de migração de banco de dados (N/A)
- Documentação técnica: README.md, este PGC e os modelos de colaboração (.github/pull_request_template.md, .github/CODEOWNERS, .github/ISSUE_TEMPLATE/funcionalidade.yml e defeito.yml)

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch de feature / correção	feature/<issue>-<descrição-curta> ou fix/<issue>-<descrição-curta>	feature/12-botao-reiniciar
Commit / título do PR	Conventional Commits: <tipo>(<escopo>): <descrição>	feat(counter): adiciona botão de reiniciar contagem
Tag de release	Versionamento Semântico vX.Y.Z	v1.0.0

Item	Convenção	Exemplo
Artefato de build	n2_gcps-<versão>.<extensão>	n2_gcps-1.0.0.apk
Issue	Modelos funcionalidade (título "feat: ") e defeito	feat: botão para reiniciar contagem

4.3 Estrutura do Repositório

```
GCPS-N2/
├── .github/
│   ├── CODEOWNERS
│   ├── pull_request_template.md
│   └── ISSUE_TEMPLATE/ (funcionalidade.yml, defeito.yml)
│
├── android/            configuração do alvo Android
├── lib/main.dart       código-fonte do app
├── test/               testes automatizados (widget_test.dart)
├── web/                configuração do alvo web
├── .gitattributes
├── .gitignore
├── .metadata
├── PGC_N2_GCPS.pdf     documento de configuração
├── README.md
├── analysis_options.yaml
├── pubspec.lock        versões exatas das dependências
└── pubspec.yaml        dependências e versão do app
```

5. Integração Contínua (CI)

Todo Pull Request para a branch principal deve disparar automaticamente um pipeline de CI. Nenhuma mudança é integrada sem que o pipeline seja concluído com sucesso. No N2_GCPS o workflow de CI será implementado na Aula 2; até lá, a proteção da main exige Pull Request e 1 aprovação.

5.1 Etapas do Pipeline de CI

1. Checkout do código e restauração de dependências (flutter pub get, com cache do pub, a partir do pubspec.lock).
2. Build da aplicação (flutter build web e/ou flutter build apk).
3. Análise estática e padrões de codificação (flutter analyze e dart format --set-exit-if-changed).
4. Testes unitários e de widget (flutter test --coverage), com medição de cobertura mínima.
5. Testes de integração — não aplicável nesta fase (o app só possui testes de widget).
6. Verificação de segurança: análise de dependências (SCA) via Dependabot e secret scanning do GitHub. O flutter analyze cobre a análise estática.
7. Empacotamento e publicação dos artefatos versionados (APK e build web) como artifacts do workflow.

5.2 Política de Pull Request e Revisão de Código

- Todo PR exige 1 aprovação de outra pessoa ("quatro olhos"). O revisor é a conta Vinicius-Batista-Condes (permissão Write). Aprovações são descartadas quando novos commits são enviados ao PR.
- PRs devem ser pequenos e tratar de um único assunto, seguindo o modelo .github/pull_request_template.md (contexto, decisão, como testar e checklist). O CODEOWNERS chama o desenvolvedor líder (@Klebynhow) para mudanças em .github/, scripts/ e pubspec.yaml.
- Só é permitido squash merge (com o título e a descrição do PR como mensagem), em histórico linear. O merge condicionado ao CI verde (quality gate) será habilitado na Aula 2, quando os checks existirem.

5.3 Critérios de Qualidade (Quality Gate)

Limites objetivos propostos, que serão configurados como status checks obrigatórios na Aula 2:

- Zero erros e avisos bloqueantes no flutter analyze.
- Código formatado (dart format sem diferenças).
- flutter test com 100% dos testes passando e cobertura de linhas ≥ 80%.
- Build web e APK concluídos sem erro.
- Nenhum segredo no código e nenhuma vulnerabilidade crítica ou alta conhecida nas dependências.

6. Entrega e Implantação Contínua (CD)

Após a publicação do artefato pelo pipeline de CI, o pipeline de CD é responsável por promover essa mesma versão, sem recompilação, através dos ambientes até a produção.

6.1 Etapas do Pipeline de CD

8. Deploy automático no environment staging após o merge na main (Aula 3).
9. Testes de aceitação em staging: smoke test do build web (a página carrega e o contador incrementa ao clicar).
10. Gate de aprovação para produção: aprovação manual obrigatória do revisor do environment production (Klebynhow).
11. Deploy em produção com a estratégia definida em 6.2.
12. Testes de fumaça pós-deploy e verificação de que a versão publicada responde.
13. Geração automática da tag de release vX.Y.Z e das notas de versão a partir dos títulos dos PRs (Aula 3).

6.2 Estratégias de Implantação

Estratégia	Descrição
Rolling update	Substituição gradual das instâncias antigas pelas novas, minimizando indisponibilidade.
Blue-Green	Duas versões (ambientes) coexistem; o tráfego é comutado de uma vez para a nova versão, permitindo rollback imediato.
Canary Release	A nova versão recebe uma pequena fração do tráfego real, sendo ampliada gradualmente conforme a confiança.
Feature Flags	Funcionalidades são implantadas desligadas e ativadas em produção de forma independente do deploy, reduzindo o risco de releases.

Estratégia adotada no N2_GCPS: substituição simples da versão publicada (equivalente a rolling update), suficiente para um app sem backend e sem usuários reais. O destino do deploy será definido na Aula 3.

6.3 Rollback e Recuperação

Toda implantação deve ser reversível. A versão anterior permanece disponível como tag, GitHub Release e artefato. O rollback é feito reexecutando o deploy da tag anterior (com a aprovação do environment production) ou revertendo o PR com um novo PR, mantendo o histórico linear.

7. Infraestrutura como Código (IaC)

Toda a infraestrutura provisionada para os ambientes de Desenvolvimento, Homologação e Produção deve ser descrita como código versionado, submetida ao mesmo fluxo de Pull Request e pipeline usado para a aplicação.

7.1 Versionamento de Infraestrutura

- O N2_GCPS não possui infraestrutura própria a provisionar. A configuração do GitHub (proteção da main, opções de merge, environments e colaboradores) é feita pela interface e documentada neste plano (seções 3.2, 3.3 e 5.2), devendo ser reaplicada conforme ele caso o repositório seja recriado.
- Se houver IaC no futuro (ex.: workflows de deploy ou Terraform), seguirá o mesmo processo de PR e revisão, e o estado ficará centralizado, nunca em máquinas locais.

7.2 Ambientes Efêmeros

Não aplicável nesta fase (app sem backend). Os builds de cada PR ficam disponíveis como artifacts do workflow de CI.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

Mudanças de rotina (funcionalidades, correções) seguem o fluxo: abertura de Issue (modelos funcionalidade ou defeito) → branch feature/<issue>-<descrição> → Pull Request com o modelo do repositório (Closes #<issue>) → revisão e aprovação do par → CI verde (a partir da Aula 2) → squash merge → CD. Esse fluxo, por si, já constitui o registro formal e rastreável da mudança.

8.2 Comitê de Controle de Mudança (CCM)

O CCM é acionado apenas em mudanças de alto impacto ou risco elevado, como alterações estruturais de infraestrutura, mudanças de esquema de banco de dados em produção, ou exceções ao fluxo padrão (ex.: hotfix emergencial). Em situações rotineiras, a aprovação por pares (revisão de PR) substitui o comitê formal.

Composição: Vinicius Batista Condes (Klebynhow, gerente do projeto e da GCS) e o revisor Vinicius-Batista-Condes. O CCM é acionado quando houver: (a) alteração em .github/ (workflows, CODEOWNERS, modelos) ou nas regras de proteção da main e dos environments; (b) atualização de dependência com mudança de versão major; (c) hotfix emergencial; (d) qualquer exceção ao fluxo padrão. Nesses casos, o autor marca o item "Mudança de alto risco" no checklist do PR.

9. Baselines e Releases

9.1 Geração Automática de Baseline

Cada implantação bem-sucedida em produção gera uma baseline: uma tag de versão (SemVer) associada ao commit na main, aos artefatos do build (APK e web) anexados à GitHub Release e à versão registrada no pubspec.yaml. A automação será implementada na Aula 3; até lá, as baselines são criadas manualmente pela página de Releases.

9.2 Notas de Versão (Release Notes)

As notas de release são geradas a partir dos títulos dos PRs (Conventional Commits) desde a última tag e publicadas na GitHub Release correspondente.

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

Por ser um app didático sem backend, não há monitoramento de runtime. A saúde das entregas é acompanhada pelo status dos workflows, pelo histórico de deployments nos environments staging e production e pelos testes de fumaça pós-deploy.

10.2 Métricas DORA

Métrica	Descrição
Deployment Frequency	Frequência com que o código é implantado em produção.
Lead Time for Changes	Tempo entre o commit e a implantação em produção.
Change Failure Rate	Percentual de implantações que resultam em falha em produção.
Mean Time to Restore (MTTR)	Tempo médio para restaurar o serviço após uma falha.

10.3 Relatórios e Auditorias

O histórico de Pull Requests (autor, revisor, aprovação), as execuções do GitHub Actions, as aprovações de deployment nos environments e as tags e releases constituem o principal insumo de auditoria de GCS, dispensando relatórios manuais.

11. Marcos

Aula 1 — Repositório configurado: itens básicos, modelos de colaboração, par de revisão, squash merge, environments staging e production, proteção da main e este PGC publicado.

Aula 2 — Pipeline de CI (analyze, format, test e build) configurado como status check obrigatório na main.

Aula 3 — Pipeline de CD (deploy em staging e gate de aprovação em production), versionamento a partir dos Conventional Commits e primeira release automatizada.

12. Treinamento e Recursos

Ferramentas: Git, GitHub (Pull Requests, Environments, Actions, Secrets), Flutter SDK e Dart, e um editor (VS Code ou Android Studio). Pessoal: o desenvolvedor/responsável pela GCS e o revisor do par. Treinamento: material das Aulas 1 a 3, documentação oficial do Flutter e do GitHub, Conventional Commits e Versionamento Semântico.

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

Bibliotecas e serviços de terceiros integrados ao produto também são itens de configuração e devem ser versionados explicitamente (arquivo de dependências travado — lockfile).

- Análise automática de vulnerabilidades em dependências (SCA) integrada ao pipeline de CI.
- Atualizações de dependências propostas automaticamente (ex.: Dependabot/Renovate) e submetidas ao mesmo fluxo de PR.

Dependências atuais (pubspec.yaml): cupertino_icons ^1.0.8 (execução); flutter_test e flutter_lints ^6.0.0 (desenvolvimento). As versões exatas são travadas no pubspec.lock, versionado no repositório. Não há subcontratados ou fornecedores externos.

14. Quadro-Resumo do Pipeline CI/CD

Resumo do fluxo padrão, do commit à produção:

Etapa	Descrição	Gatilho
CI	Build, lint, testes unitários/integração, análise de segurança, publicação do artefato/imagem.	Abertura/atualização de Pull Request
Merge	Integração da mudança na branch principal.	Aprovação de PR + CI verde
CD — Staging	Deploy automático em homologação + testes de aceitação.	Merge na branch principal
Gate de Release	Aprovação para produção (automática ou manual, conforme criticidade).	Sucesso dos testes em staging
CD — Produção	Deploy (rolling/blue-green/canary), smoke tests, geração de baseline/tag.	Aprovação do gate de release